

ar_mess.py

=====

```
import os
import c_db
import pathlib
from pathlib import Path
import threading
import time
import json
import sqlite3

class ar_directory:
    def __init__(self, project_name, directory_path=None):
        self.project_name = project_name
        self.directory_path = directory_path or os.getcwd()
        #self.file_data = self.directory()
        self.file_data = self.directory()
        #self.return_paths()

        #print(self.file_data)
        #self.fix_path()

    def fix_path(self):
        self.directory_path = self.directory_path.replace("\\", "/")

    def return_paths(self):
        return self.file_data

    def directory(self):
        # Get the current directory
        current_directory = self.directory_path or os.getcwd()

        # List all files in the directory
        files = os.listdir(current_directory)

        # Collect file details
        self.file_data = []
        for file in files:
            file_path = os.path.join(current_directory, file)
            if os.path.isfile(file_path):
                file_info = {
                    "name": file,
                    "size": os.path.getsize(file_path),
                    "last_update": pathlib.Path(file_path).stat().st_mtime,
                    "path": file_path
                }
                if file_info["name"] != self.file_data:
                    self.file_data.append(file_info)
        return self.file_data

        # Print the collected data
        for data in self.file_data:
            print(f"Name: {data['name']}, Size: {data['size']} bytes, Last Update: {data['last_update']},")
        #return self.file_data

    def filter_files(self):
        d1 = c_db.data("files_data.db", self.project_name)
        new_files = []
        for data in self.file_data:
            if not d1.is_file_record_exists(data["name"]):
                new_files.append(data)
            elif d1.is_file_record_exists(data["name"]) and d1.get_file_record(data["name"])[3] != data["last_update"]:
                new_files.append(data)
        self.file_data = new_files
        print("Filtered files len:", len(self.file_data))

    def print_all_records(self):
        d1 = c_db.data("files_data.db", self.project_name)
        d1.print_all_records()

    def add_file_record(self, name_of_file, file_size, update_date):
```

```

d1 = c_db.data("files_data.db", self.project_name)
d1.add_file_record(name_of_file, file_size, update_date)

def add_to_database(self):
    d1 = c_db.data("files_data.db", self.project_name)
    for data in self.file_data:
        d1.add_file_record(data["name"], data["size"], data["last_update"])

def clear_all_records(self):
    """
    Clear all tracked file records in local state and in the files DB.
    """
    self.file_data = []

    db_path = "files_data.db"
    d1 = c_db.data(db_path)

    if hasattr(d1, "clear_file_records"):
        d1.clear_file_records()
    elif hasattr(d1, "delete_all_files"):
        d1.delete_all_files()
    else:
        # fallback: direct sqlite delete (guess table name)
        try:
            conn = sqlite3.connect(db_path)
            cur = conn.cursor()
            cur.execute("DELETE FROM files")
            conn.commit()
        finally:
            conn.close()

    print("ar_directory: all file records cleared")

def run(self):
    #self.clear_all_records()
    self.directory()
    self.filter_files()
    #self.add_to_database()
    #d1 = c_db.data("files_data.db")
    #print("Database records:")
    #d1.print_all_records()
    #print("-----")
    return self.file_data

```

```

class DatabaseManager:
    def get_lowest_price_server(self, required_space):
        self.cursor.execute('''
            SELECT * FROM servers WHERE space >= ? ORDER BY price ASC LIMIT 1
        ''', (required_space,))
        return self.cursor.fetchone()

    def __init__(self, db_name):
        self.connection = sqlite3.connect(db_name)
        self.cursor = self.connection.cursor()
        self.create_tables()

    def create_tables(self):
        self.cursor.execute('''
            CREATE TABLE IF NOT EXISTS user_data (
                id INTEGER PRIMARY KEY,
                username TEXT NOT NULL,
                password TEXT NOT NULL,
                email TEXT NOT NULL,
                need_to_pay INTEGER NOT NULL DEFAULT 0
            )
        ''')
        self.cursor.execute('''
            CREATE TABLE IF NOT EXISTS projects (
                id INTEGER PRIMARY KEY,

```

```

        project_name TEXT NOT NULL,
        username TEXT NOT NULL,
        ip TEXT NOT NULL,
        price INTEGER NOT NULL,
        curr_space INTEGER NOT NULL,
        max_space INTEGER NOT NULL,
        path TEXT NOT NULL
    )
''' #might have problem with username in both tables
self.cursor.execute('''
    CREATE TABLE IF NOT EXISTS servers (
        id INTEGER PRIMARY KEY,
        user TEXT NOT NULL,
        user_ip TEXT NOT NULL,
        price REAL NOT NULL,
        space INTEGER NOT NULL
    )
''' #wrong need to be a different db
self.cursor.execute('''
    CREATE TABLE IF NOT EXISTS files (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        project_id INTEGER SECONDARY KEY NOT NULL,
        name_of_file TEXT NOT NULL,
        file_size INTEGER NOT NULL,
        update_date INTEGER NOT NULL
    )
'''
self.connection.commit()

def add_user(self, username, password, email, need_to_pay=0):
    self.cursor.execute('''
        INSERT INTO user_data (username, password, email, need_to_pay)
        VALUES (?, ?, ?, ?)
    ''', (username, password, email, need_to_pay))
    self.connection.commit()

def add_to_pay(self, username, amount):
    self.cursor.execute('''
        UPDATE user_data SET need_to_pay = need_to_pay + ? WHERE username = ?
    ''', (amount, username))
    self.connection.commit()

def sub_to_pay(self, username, amount):
    self.cursor.execute('''
        UPDATE user_data SET need_to_pay = need_to_pay - ? WHERE username = ?
    ''', (amount, username))
    self.connection.commit()

def add_project(self, project_name, username, ip, price, curr_space, max_space, path):
    self.cursor.execute('''
        INSERT INTO projects (project_name, username, ip, price, curr_space, max_space, path)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', (project_name, username, ip, price, curr_space, max_space, path))
    self.connection.commit()

def add_server(self, user, user_ip, price, space):
    self.cursor.execute('''
        INSERT INTO servers (user, user_ip, price, space)
        VALUES (?, ?, ?, ?)
    ''', (user, user_ip, price, space))
    self.connection.commit()

def add_file(self, project_name, name_of_file, file_size, update_date):
    project_id = self.get_project_id(project_name)
    self.cursor.execute('''
        INSERT INTO files (project_id, name_of_file, file_size, update_date)
        VALUES (?, ?, ?, ?)
    ''', (project_id, name_of_file, file_size, update_date))
    self.connection.commit()

def get_project_id(self, project_name):

```

```

        self.cursor.execute('SELECT id FROM projects WHERE project_name = ?', (project_name,))
        result = self.cursor.fetchone()
        return result[0] if result else None

def remove_user(self, user_id):
    self.cursor.execute('DELETE FROM user_data WHERE id = ?', (user_id,))
    self.connection.commit()

def remove_project(self, project_id):
    self.cursor.execute('DELETE FROM projects WHERE id = ?', (project_id,))
    self.connection.commit()

def remove_server(self, server_id):
    self.cursor.execute('DELETE FROM servers WHERE id = ?', (server_id,))
    self.connection.commit()

def user_exists(self, username):
    self.cursor.execute('SELECT * FROM user_data WHERE username = ?', (username,))
    return self.cursor.fetchone() is not None

    ger

def get_projects_list(self):
    self.cursor.execute('SELECT project_name FROM projects')
    return [row[0] for row in self.cursor.fetchall()]

def print_all_projects(self):
    self.cursor.execute('SELECT * FROM projects')
    for row in self.cursor.fetchall():
        print(row)

def get_ip_by_project(self, project_name):
    self.cursor.execute('SELECT ip FROM projects WHERE project_name = ?', (project_name,))
    result = self.cursor.fetchone()
    return result[0] if result else None

def get_project_path(self, project_name):
    self.cursor.execute('SELECT path FROM projects WHERE project_name = ?', (project_name,))
    result = self.cursor.fetchone()
    return result[0] if result else None

def close(self):
    self.connection.close()

def check_login(self, username, password):
    self.cursor.execute('SELECT * FROM user_data WHERE username = ? AND password = ?', (username, password))
    return self.cursor.fetchone() is not None

def clear_data_base(self):
    self.cursor.execute('DELETE FROM user_data')
    self.cursor.execute('DELETE FROM projects')
    self.cursor.execute('DELETE FROM servers')
    self.cursor.execute('DELETE FROM files')
    self.connection.commit()

def delete_file_record(self, project_name, name_of_file):
    project_id = self.get_project_id(project_name)
    self.cursor.execute('DELETE FROM files WHERE project_id = ? AND name_of_file = ?', (project_id, name_of_file))
    self.connection.commit()

def get_file_record(self, project_name, name_of_file):
    project_id = self.get_project_id(project_name)
    self.cursor.execute('SELECT * FROM files WHERE project_id = ? AND name_of_file = ?', (project_id, name_of_file))
    return self.cursor.fetchone()

def get_all_file_records(self, project_name):
    project_id = self.get_project_id(project_name)
    self.cursor.execute('SELECT * FROM files WHERE project_id = ?', (project_id,))
    return self.cursor.fetchall()

def is_file_record_exists(self, project_name, name_of_file):
    project_id = self.get_project_id(project_name)
    self.cursor.execute('SELECT * FROM files WHERE project_id = ? AND name_of_file = ?', (project_id, name_of_file))

```

```

        return self.cursor.fetchone() is not None
def filter_file_records(self, project_name, file_data_list):
    project_id = self.get_project_id(project_name)
    filtered_files = []
    for file_data in file_data_list:
        if not self.is_file_record_exists(project_name, file_data["name"]): #need to add if data
            filtered_files.append(file_data)
        else:
            record = self.get_file_record(project_name, file_data["name"])
            if record[3] != file_data["last_update"]:
                filtered_files.append(file_data)
    return filtered_files
def directory(self, project_name):
    """Get all files from project path with size, last change, name and path"""
    project_path = self.get_project_path(project_name)
    if not project_path:
        return []

    files = []
    try:
        for file in os.listdir(project_path):
            file_path = os.path.join(project_path, file)
            if os.path.isfile(file_path):
                files.append({
                    "name": file,
                    "size": os.path.getsize(file_path),
                    "last_update": pathlib.Path(file_path).stat().st_mtime,
                    "path": file_path
                })
    except Exception as e:
        print(f"Error reading directory: {e}")

    return files

def run(self, project_name):
    file_data = self.directory(project_name)
    filtered_files = self.filter_file_records(project_name, file_data)
    return filtered_files
class JSONConfig:
    def __init__(self, filename):
        self.filename = filename
        self.data = {"max_space": 20, "space_left": 20, "code": ""}
        self.load()

    def load(self):
        if os.path.exists(self.filename):
            with open(self.filename, 'r') as f:
                self.data = json.load(f)

    def save(self):
        with open(self.filename, 'w') as f:
            json.dump(self.data, f, indent=2)

    def set_value(self, key, value):
        self.data[key] = value
        self.save()

    def get_value(self, key):
        return self.data.get(key)

"""
# Example usage
db_manager = DatabaseManager("user_data.db")
db_manager.add_user("john_doe", "securepassword", "john@example.com", "192.168.1.1")
#db_manager.add_project("Project Alpha", 1, "192.168.1.1")
db_manager.add_server("john_doe", "192.168.1.1", 19.99, 100)
db_manager.close()
"""
def main():
    directory_path = Path(r"C:\Data\roy\school\cyber\cloud\copy")
    x = ar_directory(directory_path)

```

```

        y = x.return_paths()
        for data in y:
            print(f"Name: {data['name']}, Size: {data['size']} bytes, Last Update: {data['last_update']}, Path: {data['path']}")
    """
if __name__ == "__main__":
    d = DatabaseManager("project_data.db")
    d.clear_data_base()
    d = DatabaseManager("user_data.db")
    d.clear_data_base()
    """
#     main()

```

c_db.py

=====

```

import sqlite3
from datetime import datetime

class data:
    def __init__(self, db_name, project_name, reset=True):
        self.db_name = db_name
        self.project_name = project_name
        self.reset = reset
        self.initialize_database()

    def initialize_database(self):
        conn = sqlite3.connect(self.db_name)
        cursor = conn.cursor()
        if self.reset:
            # Drop the table if it exists
            #cursor.execute('DROP TABLE IF EXISTS files')
            cursor.execute('DROP TABLE IF EXISTS {}'.format(self.project_name))
        # Create the table again
        cursor.execute('''
            CREATE TABLE IF NOT EXISTS {project_name} (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name_of_file TEXT NOT NULL,
                file_size INTEGER NOT NULL,
                update_date INTEGER NOT NULL
            )
        '''.format(project_name=self.project_name))
        conn.commit()
        conn.close()

    # Function to add a new record to the table
    def add_file_record(self, name_of_file, file_size, update_date):
        conn = sqlite3.connect(self.db_name)
        cursor = conn.cursor()
        #update_date = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        cursor.execute('''
            INSERT INTO {project_name} (name_of_file, file_size, update_date)
            VALUES (?, ?, ?)
        ''', (name_of_file, file_size, update_date))
        conn.commit()
        for row in cursor.execute("SELECT * FROM {project_name}".format(project_name=self.project_name)).fetchall():
            print(row)
        conn.close()

    def get_file_records(self):
        conn = sqlite3.connect(self.db_name)
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM {project_name}".format(project_name=self.project_name))
        records = cursor.fetchall()
        conn.close()
        return records

    def is_file_record_exists(self, name_of_file):
        conn = sqlite3.connect(self.db_name)
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM {project_name} WHERE name_of_file=?".format(project_name=self.project_name, name_of_file=name_of_file))
        record = cursor.fetchone()

```

```

        conn.close()
        return record is not None

def get_file_record(self, name_of_file):
    conn = sqlite3.connect(self.db_name)
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM {project_name} WHERE name_of_file=?".format(project_name=self.project_name, name_of_file=name_of_file))
    record = cursor.fetchone()
    conn.close()
    return record

def delete_file_record(self, name_of_file):
    conn = sqlite3.connect(self.db_name)
    cursor = conn.cursor()
    cursor.execute("DELETE FROM {project_name} WHERE name_of_file=?".format(project_name=self.project_name, name_of_file=name_of_file))
    conn.commit()
    conn.close()

def print_all_records(self):
    conn = sqlite3.connect(self.db_name)
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM {project_name}".format(project_name=self.project_name))
    records = cursor.fetchall()
    for record in records:
        print(record)
    conn.close()

def clean_database(self):
    conn = sqlite3.connect(self.db_name)
    cursor = conn.cursor()
    cursor.execute("DELETE FROM {project_name}".format(project_name=self.project_name))
    conn.commit()
    conn.close()

def __repr__(self):
    return f"FileRecord(name_of_file={self.name_of_file}, file_size={self.file_size}, update_date={self.update_date})"

```

client.py

=====

```

import socket
import base64
import os
from ar_mess import ar_directory
import threading
from pathlib import Path
from main_menu import mange
import project_db
import time
from ar_mess import DatabaseManager as d_manager

SERVER_HOST = 'localhost'
SERVER_PORT = 5002
BUFFER_SIZE = 4096

class CloudClient:
    def __init__(self, host=SERVER_HOST, port=SERVER_PORT):
        self.host = host
        self.port = port
        self.sock = None
        self.username = None

    def connect(self):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.sock.connect((self.host, self.port))

    def disconnect(self):
        if self.sock:
            self.sock.close()
            self.sock = None

    def send_and_receive(self, message):

```

```

        self.sock.send(message.encode())
        return self.sock.recv(BUFFER_SIZE).decode()

def login(self):    #fix in server
    self.connect()
    username = input("Username: ")
    password = input("Password: ")
    response = self.send_and_receive(f"LOGIN|{username}|{password}")
    print("Server:", response)
    if response == "LOGIN_SUCCESS":
        self.username = username
        self.password = password
        return True
    self.disconnect()
    return False

def check_text(self, text):
    allowed_chars = set("abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789_")
    if len(text) == 0:
        return False
    for char in text:
        if char not in allowed_chars:
            return False
    return True

#fix max length of username and password and password

def signup(self):    #need to add email
    #self.connect()
    username = input("Choose username: ")
    password = input("Choose password: ")
    email = input("Enter your email: ")
    if self.check_text(username) == False or self.check_text(password) == False:
        print("Username or password cant be use, pls try again.")
        self.signup()
    response = self.send_and_receive(f"SIGNUP|{username}|{password}|{email}")
    print("Server:", response)
    if response == "SIGNUP_SUCCESS":
        self.username = username
        self.password = password
        return True
    self.disconnect()
    return False

#C:\Data\roy\school\cyber\cloud\copy
def project_directory(self):
    p1 = d_manager("project_data.db")
    self.db = p1
    p1.print_all_projects()
    self.project_name = input("Enter project name or type new to create a new project: ").strip()
    if self.project_name == "new":
        self.project_name = input("Enter project name: ").strip()
        friend = "y" #input("do you have an ip already? (y/n) ").strip().lower()
        if friend == "y":
            self.host = "127.0.0.1" #input("Enter the ip: ").strip()
            space = 1 #int(input("Enter the space you want to rent in GB: ").strip())
            code = 0 #int(input("Do you have a code? (Write code for yes, 0 for no): ").strip())
            path = "C:\\Data\\roy\\school\\cyber\\cloud\\copy" #input("Enter the path of the project: ")
            ok = True
            if code != 0 and ok:
                response = self.send_and_receive(f"CREATE_PROJECT|{self.project_name}|{space}|0|{code}")
                if self.signup() == False:
                    print("Signup failed, try again.")
                    self.project_directory()
                print("Server:", response)
                if response == "PROJECT_CREATED":
                    p1.add_project(self.project_name, self.username, self.host, 0,0, space, path)
            else:
                print("-----")
                self.project_directory()

```



```

        else:
            price = 1 #int(input("Enter the price you want to pay for 1 GB per month: ").strip())
            response = self.send_and_receive(f"CREATE_PROJECT|{self.project_name}|{space}|{price}")
            if self.signup() == False:
                print("Signup failed, try again.")
                self.project_directory()
            #print("Server:", response)
            #if response == "PROJECT_CREATED":
            else:
                p1.add_project(self.project_name, self.username, self.host, price,0, space, path)
            self.project_directory = path
        else:
            #rent a server from the market
            pass
    else:
        ip = p1.get_ip_by_project(self.project_name)
        if ip is not None:
            self.host = ip
            print(f"entering '{self.project_name}'")
            self.project_directory = p1.get_project_path(self.project_name)
            self.connect() #need to continue
            username = input("Username: ")
            password = input("Password: ")
            response = self.send_and_receive(f"LOGIN|{username}|{password}")
            if response == "LOGIN_SUCCESS":
                self.username = username
                self.password = password
                print(f"Successfully logged in to project '{self.project_name}' at {self.host}.")
                print("initialzing project")
                response = self.send_and_receive(f"OPEN_PROJECT|{self.project_name}")
                if response == "PROJECT_OPENED":
                    print(f"Successfully connected to project '{self.project_name}' at {self.host}.")
                else:
                    print(f"Failed to open project '{self.project_name}': {response}")
                    self.disconnect()
                    return
            else:
                print(f"Failed to log in to project '{self.project_name}': {response}")
                self.disconnect()
                self.project_directory()
        else:
            print("Project not found.")
            self.project_directory()

"""
def project_directory_2(self):
    #project_db.create_json_file()
    p1 = project_db.TextFileManager()
    response = self.send_and_receive("PROJECT_LIST")
    print("Server:", response)
    self.project_name = input("Enter project name or type New to create a new project: ").strip()
    if self.project_name.lower() == "new":
        self.project_name = input("Enter project name: ").strip()
        self.project_type = 1
        response = self.send_and_receive(f"CREATE_PROJECT|{self.project_type}|{self.project_name}")
        print("Server:", response)
        if response == "PROJECT_CREATED":
            self.project_directory = input("Enter project directory path: ").strip()
            if not os.path.exists(self.project_directory):
                print("Directory not found.")
                while True:
                    self.project_directory = Path(input("Enter project directory path: ").strip())
                    if os.path.exists(self.project_directory):
                        break
            p1.add_project(self.project_name, self.project_directory, self.project_type)
            x = ar_directory(Path(self.project_directory))
            files_data = x.return_paths()
            lis = [data["path"] for data in files_data]
            self.upload_all_files(lis)
            print(f"Project '{self.project_name}' created successfully.")

```

```

        else:
            print("Failed to create project.")

    else:
        response = self.send_and_receive(f"OPEN_PROJECT|{self.project_name}")
        print("Server:", response)
        if response == "PROJECT_NOT_FOUND":
            print("Project not found.")
            self.project_directory()
        else:
            print(f"entering '{self.project_name}'.")
            self.project_directory = p1.get_project_path(self.project_name)
        """
def upload_file(self, path):
    try:
        #filename = os.path.basename(path)
        filename = os.path.basename(path)
        with open(path, "rb") as f:
            file_bytes = f.read()
            b64 = base64.b64encode(file_bytes).decode()

        # Create a new socket for each upload to avoid threading conflicts
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as upload_sock:
            upload_sock.connect((self.host, self.port))

            # First login again to establish identity
            upload_sock.send(f"LOGIN|{self.username}|{self.password}".encode())
            response = upload_sock.recv(BUFFER_SIZE).decode().strip()
            if response != "LOGIN_SUCCESS":
                raise Exception(f"Authentication failed: {response}")

            # Send project context
            current_project = os.path.basename(self.project_directory)
            upload_sock.send(f"OPEN_PROJECT|{self.project_name}".encode())
            response = upload_sock.recv(BUFFER_SIZE).decode().strip()
            if response != "PROJECT_OPENED":
                raise Exception(f"Failed to open project context: {response}")

            #send upload command
            header = f"UPLOAD|{filename}|{len(b64)}"
            upload_sock.sendall((header).encode())

            response = upload_sock.recv(BUFFER_SIZE).decode().strip()
            if response == "READY_TO_RECEIVE":
                upload_sock.sendall(b64.encode())
                response = upload_sock.recv(BUFFER_SIZE).decode().strip()
            if response != "UPLOAD_SUCCESS":
                raise Exception(f"Upload failed: {response}")

            print(f"[{threading.current_thread().name}] Uploaded: {filename}")
            db = d_manager("project_data.db")
            db.add_file(self.project_name, filename, len(file_bytes), time.time())

    except Exception as e:
        print(f"[ERROR] Failed to upload {path}: {e}")

def upload_all_files(self, file_paths, max_threads=5):
    semaphore = threading.Semaphore(max_threads)
    threads = []

    def thread_upload(path):
        with semaphore:
            try:
                self.upload_file(path)

            except Exception as e:
                print(f"[ERROR] Failed to upload {path}: {e}")

    for path in file_paths:
        t = threading.Thread(target=thread_upload, args=(path,))

```

```

        threads.append(t)
        t.start()

    for t in threads:
        t.join()

    print("■ All uploads finished.")

def download_file(self, filename):
    try:
        # Create a new socket for each download to avoid threading conflicts
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as download_sock:
            download_sock.connect((self.host, self.port))

            # First login again to establish identity
            download_sock.send(f"LOGIN|{self.username}|{self.password}".encode())
            response = download_sock.recv(BUFFER_SIZE).decode().strip()
            if response != "LOGIN_SUCCESS":
                raise Exception(f"Authentication failed: {response}")

            # Send project context
            download_sock.send(f"OPEN_PROJECT|{self.project_name}".encode())
            response = download_sock.recv(BUFFER_SIZE).decode().strip()
            if response != "PROJECT_OPENED":
                raise Exception(f"Failed to open project context: {response}")

            # Send download command
            download_sock.send(f"DOWNLOAD|{filename}".encode())
            file_data = download_sock.recv(BUFFER_SIZE)

            if file_data:
                with open(filename, "wb") as f:
                    f.write(file_data)
                print(f"Downloaded: {filename}")
            else:
                print(f"Failed to download: {filename}")

    except Exception as e:
        print(f"[ERROR] Failed to download {filename}: {e}")

def download_project(self):
    msg = f"DOWNLOAD_PROJECT|{self.project_name}"
    response = self.send_and_receive(msg)
    print("Server:", response)
    if response.startswith("DOWNLOAD"):
        _, size_str = response.split("|")
        expected_size = int(size_str)
        self.sock.send(b"READY")
        received_data = b""
        while len(received_data) < expected_size:
            chunk = self.sock.recv(BUFFER_SIZE)
            if not chunk:
                break
            received_data += chunk

        if len(received_data) == expected_size:
            zip_data = base64.b64decode(received_data)
            with open(f"{self.project_name}.zip", "wb") as f:
                f.write(zip_data)
            print(f"Project '{self.project_name}' downloaded successfully.")
        else:
            print(f"Failed to download project: Expected {expected_size} bytes, received {len(received_data)} bytes")

def run(self):
    self.connect()
    """
    while True:

```

```

where = input("do you want to manage your server or your project? (manage/projects) ").lower()
if where == "manage":
    m1 = mange()
    m1.choose()
    break
    continue
break
#need to first gives a list of projects

action = input("Login or Signup? ").lower()
if action == "login" and self.login():
    break
elif action == "signup" and self.signup():
    break
else:
    print("Try again.")
    ""
print(f"Welcome, {self.username}!")
self.project_directory()

while True:
    cmd = input("Type 'upload' to send a file, 'upload all', 'download file', 'download all' or 'quit' ")
    if cmd == "upload":
        path = input("Enter full path to file: ")
        self.upload_file(path)
    elif cmd == "upload all":
        #d1 = ar_directory(self.project_name, Path(self.project_directory))
        #d1.print_all_records()
        #files = d1.run()
        #file_paths = [file['path'] for file in files]
        #self.upload_all_files(file_paths, d1)
        files = self.db.run(self.project_name)
        file_paths = [file['path'] for file in files]
        self.upload_all_files(file_paths)
        print("uploading finished.")
        #d1.print_all_records()
    elif cmd == "download":
        filename = input("Enter filename to download: ")
        self.download_file(filename)
    elif cmd == "download all":
        self.download_project()
    elif cmd == "quit":
        break
    else:
        print("Unknown command.")

self.disconnect()

if __name__ == "__main__":
    client = CloudClient()
    client.run()

```

generate_pdf.py

=====

```

import os
from reportlab.platypus import SimpleDocTemplate, Preformatted, Spacer
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.pagesizes import A4

```

```

def build_combined_pdf(py_files, output_file):
    code_style = ParagraphStyle(
        name="CodeStyle",
        fontName="Courier",
        fontSize=9,
        leading=11,
        leftIndent=0,
        rightIndent=0,
        firstLineIndent=0,
        spaceBefore=0,

```

```

        spaceAfter=0,
    )

    header_style = ParagraphStyle(
        name="HeaderStyle",
        fontName="Courier-Bold",
        fontSize=11,
        leading=13,
        leftIndent=0,
        rightIndent=0,
        firstLineIndent=0,
        spaceBefore=0,
        spaceAfter=0,
    )

    doc = SimpleDocTemplate(
        output_file,
        pagesize=A4,
        leftMargin=20,
        rightMargin=20,
        topMargin=20,
        bottomMargin=20,
    )

    story = []

    for index, py_file in enumerate(py_files):
        with open(py_file, "r", encoding="utf-8") as f:
            text = f.read().expandtabs(4)

            header_text = f"{os.path.basename(py_file)}\n======"

            story.append(Preformatted(header_text, header_style))
            story.append(Spacer(1, 8))
            story.append(Preformatted(text, code_style))

            if index != len(py_files) - 1:
                story.append(Spacer(1, 16))

    doc.build(story)

def main():
    current_dir = os.getcwd()
    output_dir = os.path.join(current_dir, "res_pdf_2")
    os.makedirs(output_dir, exist_ok=True)

    py_files = sorted(
        [
            os.path.join(current_dir, f)
            for f in os.listdir(current_dir)
            if f.endswith(".py")
        ]
    )

    if not py_files:
        print("No .py files found in current directory.")
        return

    output_file = os.path.join(output_dir, "combined_python_files.pdf")

    try:
        build_combined_pdf(py_files, output_file)
        print(f"Created combined PDF: {output_file}")
    except Exception as e:
        print(f"Failed to create PDF: {e}")

if __name__ == "__main__":
    main()

```

main.py

=====

```
import client
import ar_mess
import c_db
import os
import pathlib
import socket
import base64
import time
import threading
import datetime
import json
```

main_menu.py

=====

```
import socket
import base64
```

```
SERVER_HOST = 'localhost' # Change to your server's IP if not running locally
SERVER_PORT = 5002
BUFFER_SIZE = 4096
```

```
class mange:
    def __init__(self):
        pass
    def choose(self):
        c = input("do you want to rent or buy a server? (rent/buy) ")
        if c.lower() == "rent":
            self.rent()
        elif c.lower() == "buy":
            self.buy()

    def rent(self):
        sure = input("do you want to rent a server? (y/n) ")
        if sure.lower() == "y":
            space_gb = int(input("how many space in GB do you want to rent? "))
            price = int(input("enter the amount of money you want to charge for 1 GB per month: "))
            ip = input("enter the IP address of the server you want to rent: ")
            self.send_command(f"RENT|{space_gb}|{price}|{ip}") #You are renting a server with {space_gb}

    def buy(self):
        sure = input("do you want to buy a server? (y/n) ")
        if sure.lower() == "y":
            space_gb = int(input("how many space in GB do you want to buy? "))
            price = int(input("enter the amount of money you want for 1 GB: "))
            self.send_command(f"BUY|{space_gb}|{price}") #You are buying a server with {space_gb} GB for

    def send_command(self, cmd):
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
            s.connect((SERVER_HOST, SERVER_PORT))
            s.send(cmd.encode())
            response = s.recv(BUFFER_SIZE).decode()
            print(f"Server response: {response}")

    def run(self):
        pass
```

project_db.py

=====

```
import json
import os
class TextFileManager:
```

```

def __init__(self, filename="project_men.json"):
    self.filename = filename
    self.data = {}
    self._ensure_file_exists()

def _ensure_file_exists(self):
    if not os.path.exists(self.filename):
        with open(self.filename, "w") as f:
            json.dump(self.data, f, indent=4)
    self.data = self.load_data()

def load_data(self):
    with open(self.filename, "r") as f:
        return json.load(f)

def save_data(self):
    with open(self.filename, "w") as f:
        json.dump(self.data, f, indent=4)

def add_project(self, project_name, project_path, project_type):
    if project_path in self.data:
        print("Project already exists.")
    else:
        self.data[project_name] = {"path" : project_path , "type" : project_type}
        self.save_data()
        print("Project added.")

def get_project_path(self, project_name):
    self.data = self.load_data() # error here forgot to write self
    return self.data[project_name]["path"]

```

```
# {name : {path: type}}
```

server.py

```
=====
```

```

import socket
import base64
import os
import threading
#from user_database import check_login, add_user, user_exists
from ar_mess import DatabaseManager as d_manager
from ar_mess import JSONConfig as j_config

```

```

HOST = '0.0.0.0'
PORT = 5002
BUFFER_SIZE = 4096
USER_ROOT = "user_data" # top-level directory for all users

```

```

# ensure user data root exists
os.makedirs(USER_ROOT, exist_ok=True)

```

```

def ensure_user_dir(username):
    path = os.path.join(USER_ROOT, username)
    os.makedirs(path, exist_ok=True)
    return path

```

```

def handle_client(conn, addr):
    print(f"[+] Connected: {addr}")
    try:
        username = None
        project_path = None

        user_dir = None
        msg = conn.recv(BUFFER_SIZE).decode()
        cmd, *parts = msg.strip().split("|")
        db_u = d_manager("user_data.db")
    
```

```

if cmd == "LOGIN":
    username, password = parts
    if not db_u.check_login(username, password):
        return conn.send(b"LOGIN_FAIL")
    conn.send(b"LOGIN_SUCCESS")
    user_dir = ensure_user_dir(username)

#elif cmd.startswith("CREATE_PROJECT"):
elif cmd == "CREATE_PROJECT":
    conn.send(b"bla")
    msg_2 = conn.recv(BUFFER_SIZE).decode()
    cmd_2, *parts_2 = msg_2.strip().split("|")
    all_good = True
    if cmd_2 == "SIGNUP":
        #really not sure
        username, password, email = parts_2
        if db_u.user_exists(username):
            conn.send(b"USERNAME_EXISTS")
            #all_good = False
        db_u.add_user(username, password, email)
        user_dir = ensure_user_dir(username)
        conn.send(b"SIGNUP_SUCCESS")
    else:
        conn.send(b"ERROR: AUTH REQUIRED")
        #all_good = False
    #_, project_name, ip, space, price, code = msg.split("|") #why do i send the ip
    _, project_name, space, price, code = msg.split("|") #why do i send the ip
    project_path = os.path.join(user_dir, project_name)
    if os.path.exists(project_path):
        conn.send(b"PROJECT_EXISTS")
    else:
        if code == "0":
            db_u.add_to_pay(username, price)
            j_config("config.json").set_value("space_left", j_config("config.json").get_value("space_left") + price)
        elif code != j_config("config.json").get_value("code"):
            conn.send(b"INVALID_CODE")
            all_good = False
        if int(space) > j_config("config.json").get_value("space_left"):
            conn.send(b"NOT_ENOUGH_SPACE")
            all_good = False
        if all_good:
            os.makedirs(project_path)
            conn.send(b"PROJECT_CREATED")
        else:
            conn.send(b"PROJECT_CREATION_FAILED")

else:
    return conn.send(b"ERROR: AUTH REQUIRED")

while True:
    msg = conn.recv(BUFFER_SIZE).decode()
    if not msg:
        break

    if msg.startswith("PROJECT_LIST"):
        #dosnt need to happen
        projects = os.listdir(user_dir)
        if not projects:
            conn.send(b"NO_PROJECTS")
        else:
            conn.send("".join(projects).encode())

    #signup was here

    elif msg.startswith("OPEN_PROJECT"):
        _, project_name = msg.split("|")
        project_path = os.path.join(user_dir, project_name)
        if not os.path.exists(project_path):
            conn.send(b"PROJECT_NOT_FOUND")
        else:
            conn.send(b"PROJECT_OPENED")
    elif msg.startswith("UPLOAD"):
        try:

```



```

# Make sure we have an active project
if project_path is None:
    conn.send(b"ERROR|NO_ACTIVE_PROJECT")
    continue

# header: UPLOAD|filename|size
header_parts = msg.split("|")
if len(header_parts) != 3:
    conn.send(b"ERROR|INVALID_UPLOAD_HEADER")
    continue

_, filename, b64size = header_parts
b64size = int(b64size)
if b64size > 0:
    conn.send(b"READY_TO_RECEIVE")

received = b""
while len(received) < b64size:
    chunk = conn.recv(min(BUFFER_SIZE, b64size - len(received)))
    if not chunk:
        break
    received += chunk

file_bytes = base64.b64decode(received)
save_path = os.path.join(project_path, filename)

# Ensure directory exists
os.makedirs(os.path.dirname(save_path), exist_ok=True)

with open(save_path, "wb") as f:
    f.write(file_bytes)

conn.send(b"UPLOAD_SUCCESS")
print(f"[+] File '{filename}' uploaded to {project_path}") # cmd|par1|par2

except Exception as e:
    print(f"[UPLOAD ERROR] {e}")
    conn.send(f"ERROR|UPLOAD_FAILED: {str(e)}.encode()")

elif msg.startswith("DOWNLOAD|"):
    try:
        _, filename = msg.strip().split("|")
        if project_path is None:
            conn.send(b"ERROR|NO_ACTIVE_PROJECT")
            continue

        file_path = os.path.join(project_path, filename)
        if not os.path.exists(file_path):
            conn.send(b"ERROR|FILE_NOT_FOUND")
            continue

        with open(file_path, "rb") as f:
            b64 = base64.b64encode(f.read())
            size = len(b64)
            conn.send(f"DOWNLOAD|{size}.encode()")
            ack = conn.recv(BUFFER_SIZE)
            if ack == b"READY":
                conn.sendall(b64)
    except Exception as e:
        print(f"[DOWNLOAD ERROR] {e}")
        conn.send(b"ERROR|DOWNLOAD_FAILED")

elif msg.startswith("DOWNLOAD_PROJECT"):
    try:
        _, project_name = msg.strip().split("|")
        if project_path is None:
            conn.send(b"ERROR|NO_ACTIVE_PROJECT")
            continue

        project_path = os.path.join(user_dir, project_name)
        if not os.path.exists(project_path):

```

```

        conn.send(b"ERROR|PROJECT_NOT_FOUND")
        continue

    # Create a zip of the project
    import io
    import zipfile

    zip_buffer = io.BytesIO()
    with zipfile.ZipFile(zip_buffer, 'w', zipfile.ZIP_DEFLATED) as zip_file:
        for root, _, files in os.walk(project_path):
            for file in files:
                file_path = os.path.join(root, file)
                arcname = os.path.relpath(file_path, project_path)
                zip_file.write(file_path, arcname)

    b64_zip = base64.b64encode(zip_buffer.getvalue())
    size = len(b64_zip)
    conn.send(f"DOWNLOAD|{size}".encode())
    ack = conn.recv(BUFFER_SIZE)
    if ack == b"READY":
        conn.sendall(b64_zip)
    except Exception as e:
        print(f"[DOWNLOAD PROJECT ERROR] {e}")
        conn.send(b"ERROR|DOWNLOAD_PROJECT_FAILED")
    else:
        conn.send(b"UNKNOWN_COMMAND")

except Exception as e:
    print(f"[SERVER ERROR] {e}")
    try:
        conn.send(f"ERROR|{str(e)}".encode())
    except:
        pass
finally:
    conn.close()
    print(f"[-] Disconnected: {addr}")

```

```

def start_server():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.bind((HOST, PORT))
        s.listen()
        print(f"[+] Server listening on {HOST}:{PORT}")
        while True:
            conn, addr = s.accept()
            threading.Thread(target=handle_client, args=(conn, addr), daemon=True).start()

if __name__ == "__main__":
    start_server()

```

user_database.py

=====

```

import json
import os

```

```

DB_FILE = "users.json"

```

```

def load_users():
    if not os.path.exists(DB_FILE):
        return {}
    with open(DB_FILE, "r") as f:
        return json.load(f)

```

```

def save_users(users):
    with open(DB_FILE, "w") as f:
        json.dump(users, f)

```

```
def user_exists(username):
    users = load_users()
    return username in users

def add_user(username, password):
    users = load_users()
    if username in users:
        return False
    users[username] = {"password": password}
    save_users(users)
    return True

def check_login(username, password):
    users = load_users()
    return users.get(username, {}).get("password") == password
```